Architectural & Product Decision Log (ADR) — WariMitra

Document Identifier: WM-DEC-019

Classification: Architecture Decision Records (ADR) & Product Trade-offs

Standard: Michael Nygard ADR Template / BABOK v3

1. Key Architectural & Product Decisions

ADR-01: Adoption of Firebase Serverless over Django Monolith

- **Context:** The prototype backend (Django 6.0 + PostgreSQL + Redis) faced severe connection pool exhaustion and database locks under high-frequency GPS ping loads (`Reference files/backend_audit.md`).
- **Decision:** Re-architect backend as serverless Node.js 20 Cloud Functions backed by Cloud Firestore in `asia-south1`.
- **Consequences (Positive):** Auto-scaling to 10,000+ concurrent connections; built-in real-time listeners; eliminated server administration; reduced projected operating costs to ~$185/month.
- **Consequences (Negative / Trade-offs):** Requires strict NoSQL query modeling; dynamic multi-field filtering constrained by composite index requirements.

ADR-02: Asynchronous 0-ETH Self-Transactions for Web3 Trust Layer

- **Context:** The system required tamper-evident, non-repudiable audit logs of emergency SOS incidents to withstand judicial scrutiny during disaster inquiries. However, state regulations prohibit government expenditure on volatile cryptocurrency gas fees (`docs/WEB3_ARCHITECTURE.md`).
- **Decision:** Implement asynchronous background anchoring in Cloud Functions where a 0-ETH transaction is signed to itself on the Ethereum Sepolia testnet, packing the canonical SHA-256 event hash into the transaction's hex `data` payload.
- **Consequences (Positive):** ₹0 recurring gas expenditure; permanent blockchain immutability; zero impact on emergency response latency (operates strictly in background).
- **Consequences (Negative / Trade-offs):** Relies on public testnet stability; mitigated via automated

- **Consequences (Negative / Trade-offs):** Relies on public testnet stability; mitigated via automated fallback to local Firestore collection with status `LOCAL_PROOF_ONLY`.

ADR-03: Geohash Spatial Indexing instead of Heavy PostGIS Extension

- **Context:** The mobile app requires finding the nearest active medical camps within 10 km.
- **Decision:** Store 9-character Geohashes with `geofire-common` in Firestore documents, executing bounding-box queries (`startAt` / `endAt`) and pruning with client/function Haversine formulas.
- **Consequences (Positive):** Blazing fast query execution (320ms p95); works natively within serverless Firestore without managing a dedicated PostGIS instance.
- **Consequences (Negative / Trade-offs):** Requires client or function-side distance recalculation and sorting; bounding boxes may return edge-case false positives that must be filtered in memory.

ADR-04: Virtualized List Rendering (`react-window`) for Dispatcher Queues

- **Context:** Under peak crowd incidents, the web incident feed could receive hundreds of concurrent alerts, causing DOM tree bloat and browser freezes during critical dispatch operations.
- **Decision:** Implement `react-window` list virtualization in `frontend/src/app/sos/page.tsx`.
- **Consequences (Positive):** Stable 60 FPS scrolling and low memory usage regardless of whether 10 or 1,000 incidents are active.